```
#PRN: 23070521037
#NAME: Chaitanya Kharpate
#Section: A

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
data = pd.read_csv("customer_churn.csv")

# Display first 5 rows
data.head()
```

	CustomerID	Age	Gender	Tenure	MonthlyCharges	TotalCharges	Contract	InternetService	SupportCalls	PaymentDelay	Chur
0	1001	25	Male	3	59.99	180.00	Month-to-month	Fiber	4	5	
1	1002	42	Female	36	79.99	2879.64	Two year	DSL	1	0	
2	1003	31	Male	12	65.50	786.00	One year	Fiber	2	1	
3	1004	55	Female	2	95.00	190.00	Month-to-month	Fiber	5	10	

```
# Remove CustomerID
data = data.drop("CustomerID", axis=1)

# Encode categorical columns
categorical_columns = ["Gender", "Contract", "InternetService"]

label_encoder = LabelEncoder()

for col in categorical_columns:
    data[col] = label_encoder.fit_transform(data[col])

# Features and target
X = data.drop("Churn", axis=1)
y = data["Churn"]

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Preprocessing Complete")
```

```
Preprocessing Complete
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Samples: 16
Testing Samples: 4
```

```
model = Sequential()

model.add(Dense(32, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(16, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	320
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 8)	136
dense_3 (Dense)	(None, 1)	9

Total params: 993 (3.88 KB)
Trainable params: 993 (3.88 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

print("Model Compiled Successfully")
```

```
Model Compiled Successfully
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 131ms/step - accuracy: 0.3333 - loss: 0.7064 - val_accuracy: 0.2500 - val_loss: 0.7715
Epoch 2/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.4167 - loss: 0.6832 - val_accuracy: 0.5000 - val_loss: 0.7394
Epoch 3/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5833 - loss: 0.6652 - val_accuracy: 0.5000 - val_loss: 0.7121
Epoch 4/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8333 - loss: 0.6480 - val_accuracy: 0.7500 - val_loss: 0.6879
Epoch 5/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.8333 - loss: 0.6353 - val_accuracy: 0.7500 - val_loss: 0.6611
Epoch 6/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9167 - loss: 0.6191 - val_accuracy: 0.7500 - val_loss: 0.6407
Epoch 7/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9167 - loss: 0.6041 - val_accuracy: 0.7500 - val_loss: 0.6212
Epoch 8/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9167 - loss: 0.5918 - val_accuracy: 0.7500 - val_loss: 0.6026
Epoch 9/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9167 - loss: 0.5781 - val_accuracy: 0.7500 - val_loss: 0.5854
Epoch 10/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - accuracy: 1.0000 - loss: 0.5675 - val_accuracy: 0.7500 - val_loss: 0.5713
Epoch 11/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 155ms/step - accuracy: 1.0000 - loss: 0.5548 - val_accuracy: 1.0000 - val_loss: 0.5569
Epoch 12/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - accuracy: 1.0000 - loss: 0.5442 - val_accuracy: 1.0000 - val_loss: 0.5431
Epoch 13/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 152ms/step - accuracy: 1.0000 - loss: 0.5347 - val_accuracy: 1.0000 - val_loss: 0.5308
Epoch 14/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 158ms/step - accuracy: 1.0000 - loss: 0.5219 - val_accuracy: 1.0000 - val_loss: 0.5192
Epoch 15/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - accuracy: 1.0000 - loss: 0.5056 - val_accuracy: 1.0000 - val_loss: 0.5050
Epoch 16/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 169ms/step - accuracy: 1.0000 - loss: 0.4899 - val_accuracy: 1.0000 - val_loss: 0.4880
Epoch 17/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 133ms/step - accuracy: 1.0000 - loss: 0.4663 - val_accuracy: 1.0000 - val_loss: 0.4668
Epoch 18/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step - accuracy: 1.0000 - loss: 0.4414 - val_accuracy: 1.0000 - val_loss: 0.4450
Epoch 19/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - accuracy: 1.0000 - loss: 0.4140 - val_accuracy: 1.0000 - val_loss: 0.4252
Epoch 20/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 82ms/step - accuracy: 1.0000 - loss: 0.3792 - val_accuracy: 1.0000 - val_loss: 0.4049
Epoch 21/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 108ms/step - accuracy: 1.0000 - loss: 0.3525 - val_accuracy: 1.0000 - val_loss: 0.3854
Epoch 22/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 76ms/step - accuracy: 1.0000 - loss: 0.3253 - val_accuracy: 1.0000 - val_loss: 0.3674
Epoch 23/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step - accuracy: 1.0000 - loss: 0.3007 - val_accuracy: 1.0000 - val_loss: 0.3481
Epoch 24/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 1.0000 - loss: 0.2771 - val_accuracy: 1.0000 - val_loss: 0.3290
Epoch 25/50
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - accuracy: 1.0000 - loss: 0.2556 - val_accuracy: 1.0000 - val_loss: 0.3113
Epoch 26/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 1.0000 - loss: 0.2359 - val_accuracy: 1.0000 - val_loss: 0.2950
Epoch 27/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - accuracy: 1.0000 - loss: 0.2188 - val_accuracy: 1.0000 - val_loss: 0.2794
Epoch 28/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 1.0000 - loss: 0.2031 - val_accuracy: 1.0000 - val_loss: 0.2641
Epoch 29/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - accuracy: 1.0000 - loss: 0.1887 - val_accuracy: 1.0000 - val_loss: 0.2486
```

```
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 206ms/step - accuracy: 1.0000 - loss: 0.0504
Test Loss: 0.050392553210258484
Test Accuracy: 1.0
```

```
predictions = model.predict(X_test)

predictions = (predictions > 0.5).astype(int)

print("Predicted Values:")
print(predictions.flatten())

print("\nActual Values:")
print(y_test.values)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step
Predicted Values:
[1 0 0 0]

Actual Values:
[1 0 0 0]
```

```
print("Accuracy:", accuracy_score(y_test, predictions))

print("\nClassification Report")
print(classification_report(y_test, predictions))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, predictions))
```

```
Accuracy: 1.0

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         3
           1       1.00      1.00      1.00         1

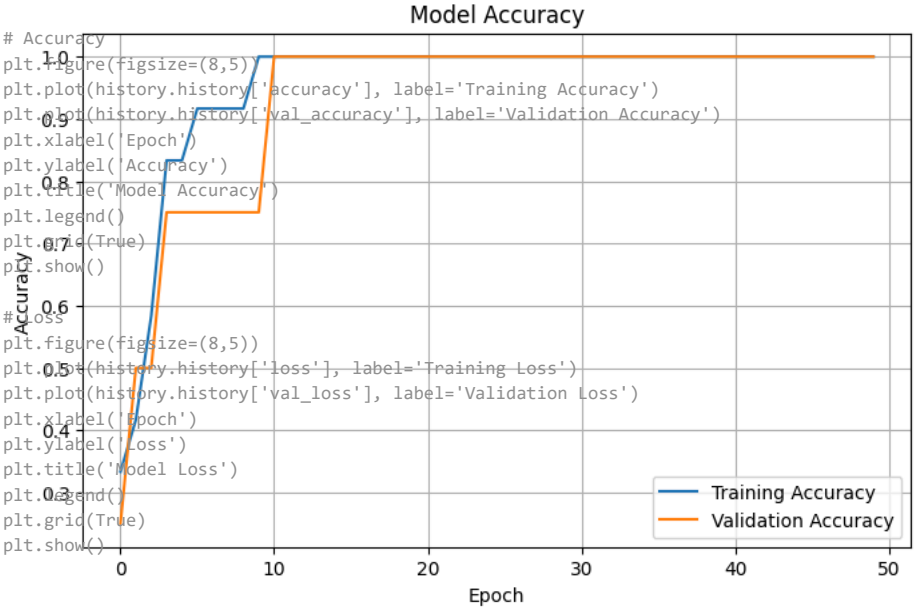
    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4

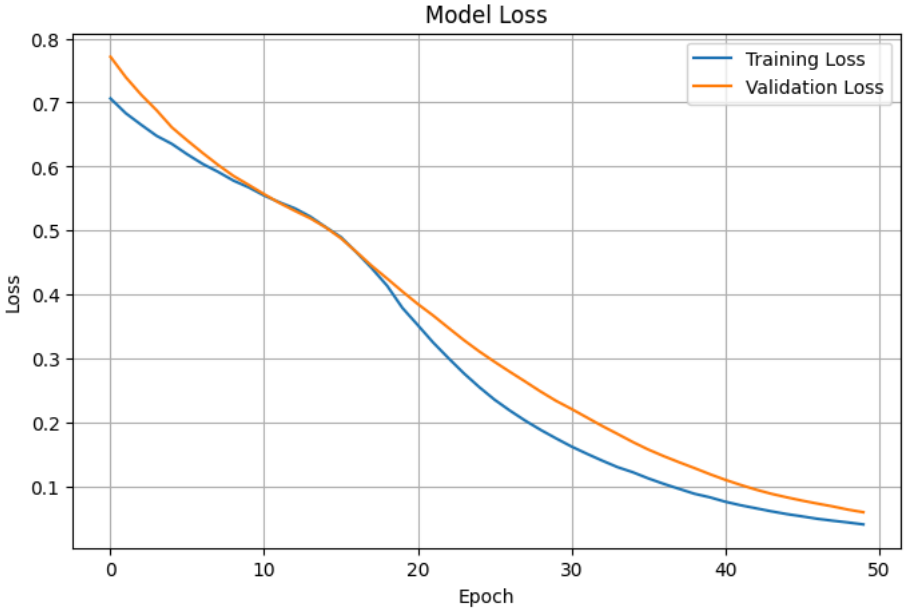

Confusion Matrix
[[3 0]
 [0 1]]
```

```
import matplotlib.pyplot as plt

# Accuracy
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Model Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# Loss
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()
plt.grid(True)
plt.show()
```





Start coding or generate with AI.